

Starter Deck Sync

Web Server Implementation Guide

Wisdom and Chance: Adding Default Starter Decks for All New Accounts

Overview

The mobile app now automatically creates 5 starter decks (one per commander) when a user first logs in and has 0 existing decks. Since all decks are stored on the shared server, these decks are already visible on the web version for users who first log in via mobile.

However, users who first log in through the web version will not get starter decks until they also open the mobile app. To ensure ALL users get starter decks regardless of which platform they log in from first, add the same seeding logic to the web server.

What the Mobile App Does

After login or session restore, the mobile app:

1. Calls GET `/api/user-decks` to check existing decks
2. If the user has 0 decks, creates 5 starter decks via POST `/api/user-decks`
3. Each deck contains 40 cards — all 4 variants of each power level (1-10) for that element

Option A: Server-Side Seeding (Recommended)

Add starter deck creation directly in the server's login/registration handler. This is the cleanest approach because it guarantees every new user gets decks regardless of client.

Step 1: Create a Starter Deck Seeder Function

Add this function to your server code (e.g., `server/starter-decks.ts`):

```

const STARTER_DECKS = [
  { name: "Pyros's Inferno",      commanderId: "commander-fire",  element: "fire" },
  { name: "Aquara's Depths",     commanderId: "commander-water", element: "water" },
  { name: "Terran's Fortress",   commanderId: "commander-earth", element: "earth" },
  { name: "Zephyros's Storm",    commanderId: "commander-air",   element: "air" },
  { name: "Gaia's Grove",        commanderId: "commander-nature", element: "nature" },
];

function buildCardIds(element: string): string[] {
  const ids: string[] = [];
  for (let power = 1; power <= 10; power++) {
    for (let variant = 0; variant < 4; variant++) {
      ids.push(`card-${element}-${power}-${variant}`);
    }
  }
  return ids; // 40 cards total
}

async function seedStarterDecks(userId: string, db: any): Promise<void> {
  const existingDecks = await db.query.userDecks.findMany({
    where: eq(userDecks.userId, userId), limit: 1,
  });
  if (existingDecks.length > 0) return;

  const deckInserts = STARTER_DECKS.map((deck) => ({
    userId, name: deck.name, commanderId: deck.commanderId,
    cardIds: buildCardIds(deck.element),
  }));
  await db.insert(userDecks).values(deckInserts);
}

```

Step 2: Call It After Login/Registration

In your login or registration route handler, add:

```

// After successful login/registration:
try {
  await seedStarterDecks(user.id, db);
} catch (error) {
  console.warn("Failed to seed starter decks:", error);
}

```

Where to Add It

Look for your login handler — likely in:

- server/routes.ts or server/auth.ts
- The route that handles POST /api/mobile/auth/login
- Also the Google OAuth callback handler

Add the seedStarterDecks() call right after the user record is created or authenticated, before sending the response.

Option B: Web Client-Side Seeding

If you prefer not to modify the server, add the same logic to the web frontend (similar to what the mobile app does). Call this function in your auth context after login succeeds and after session restore:

```
async function seedStarterDecks(): Promise<void> {
  try {
    const response = await fetch("/api/user-decks", { credentials: "include" });
    const decks = await response.json();
    if (decks.length > 0) return;

    const elements = ["fire", "water", "earth", "air", "nature"];
    const commanders = {
      fire: { id: "commander-fire", name: "Pyros's Inferno" },
      water: { id: "commander-water", name: "Aquara's Depths" },
      earth: { id: "commander-earth", name: "Terran's Fortress" },
      air: { id: "commander-air", name: "Zephyros's Storm" },
      nature: { id: "commander-nature", name: "Gaia's Grove" },
    };

    await Promise.allSettled(elements.map((el) => {
      const cardIds = [];
      for (let p = 1; p <= 10; p++)
        for (let v = 0; v < 4; v++)
          cardIds.push(`card-${el}-${p}-${v}`);
      return fetch("/api/user-decks", {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        credentials: "include",
        body: JSON.stringify({ name: commanders[el].name,
          commanderId: commanders[el].id, cardIds }),
      });
    }));
  } catch (e) { console.warn("Could not seed starter decks:", e); }
}
```

Deck Specifications

Each starter deck follows the game's deck-building rules:

Deck Name	Commander	Element	Cards
Pyros's Inferno	commander-fire	Fire	40
Aquara's Depths	commander-water	Water	40
Terran's Fortress	commander-earth	Earth	40
Zephyros's Storm	commander-air	Air	40
Gaia's Grove	commander-nature	Nature	40

Card ID Pattern

Each deck uses all

40 cards of its
element:

- Format: card-
{element}-{power}-
{variant}
- Powers: 1
through 10
- Variants: 0
through 3
- Example: card-
fire-1-0, card-
fire-1-1, ..., card-
fire-10-3

Validation Rules Met

- Exactly 40
cards per deck
- Exactly 4 cards
per power rank (1-10)
- Maximum 3
copies of any card
(all unique, so 1
copy each)

Behavior Notes

- Starter decks
are only created
when the user has 0
existing decks
- Users can
freely edit or delete
starter decks
- If a user
deletes all their
decks, starter decks
will be re-created on
next login
- The seeding is
non-blocking — login
succeeds even if
deck creation fails
- Both mobile
and web share the
same server
database, so decks
created from either

platform are visible everywhere

Quick Test

To verify starter decks work:

1. Create a new account (or use one with 0 decks)
2. Log in from the web version
3. Navigate to the Decks page
4. You should see all 5 starter decks

To test via API:

```
# Login
curl -X POST https://wisdom-and-chance-2.replit.app/api/mobile/auth/login \
  -H "Content-Type: application/json" \
  -d '{"email":"newuser@example.com","firstName":"New","lastName":"User"}'

# Check decks (use token from login response)
curl -H "Authorization: Bearer YOUR_TOKEN" \
  https://wisdom-and-chance-2.replit.app/api/user-decks
```